```
================================================================================
FULL CONVERSATION EXPORT
Topic: MOP-Triggered Reliability Agent — Based on "Beyond pass@1" Paper
================================================================================

--------------------------------------------------------------------------------
PAPER: Beyond pass@1: A Reliability Science Framework for Long-Horizon LLM Agents
Authors: Aaditya Khanal, Yangyang Tao, Junxiu Zhou — Northern Kentucky University
Date: April 1, 2026 | arXiv:2603.29231v1
--------------------------------------------------------------------------------


================================================================================
SECTION 1: WHAT THE PAPER IS ABOUT
================================================================================

The paper argues that the AI community is measuring the wrong thing when
evaluating LLM agents.

Currently, almost all benchmarks measure CAPABILITY — can the model solve a
task once? They report a metric called pass@1. But in real production systems,
agents run thousands of times on tasks that take minutes to hours. What actually
matters is RELIABILITY — does the model consistently succeed?

The authors ran ~23,000 experiments across 10 open-source models and discovered
some genuinely surprising things.

THE CORE FINDING:
As tasks get longer, model performance drops faster than you'd mathematically
expect. It's not just "harder tasks = lower scores" — errors compound and
snowball because when a model goes wrong early, it tends to stay wrong.

THE COUNTERINTUITIVE FINDINGS:

1. High variance is actually good — the best models have the most unpredictable
   long-task results, because they sometimes nail hard tasks and sometimes
   spiral. Weak models are consistently bad, so they have low variance.

2. Memory scaffolds make things worse — adding an "episodic scratchpad" to help
   agents remember things actually hurt performance on long tasks for 6 out of
   10 models. The overhead eats into their step budget.

3. Better models "melt down" more — the top frontier models (DeepSeek V3,
   MiniMax M2.5) had the highest "meltdown" rates (19%, 13%) because they
   attempt ambitious multi-step strategies that spiral when they fail. Weak
   models never even try.

4. Domain matters more than model size — document processing tasks stayed nearly
```

```
   flat across task lengths, while software engineering tasks collapsed by 2x.
   The type of task predicts reliability better than the model's parameter count.

THE 4 NEW METRICS INTRODUCED:

1. RDC (Reliability Decay Curve)
   How does pass rate change as tasks get longer?

2. VAF (Variance Amplification Factor)
   Does task length make outcomes more unpredictable?

3. GDS (Graceful Degradation Score)
   Instead of pass/fail, how much of the task did the agent actually complete?

4. MOP (Meltdown Onset Point)
   At exactly what step does an agent start "losing its mind"
   (looping, contradicting itself)?

KEY DATA POINTS FROM THE PAPER:
- 396 tasks across 4 duration buckets and 3 domains
- 10 open-source models evaluated
- 23,392 total episodes (k=3 repeats, two scaffolds)
- Aggregate pass@1 drops from 76.3% (short) to 52.1% (very long)
- SE domain GDS: 0.90 (short) → 0.44 (very long)
- DP domain GDS: 0.74 (short) → 0.71 (very long) — nearly flat
- Frontier cluster (DeepSeek V3, Kimi K2.5, MiniMax M2.5): VL pass@1 ⩾ 79.8%
- Memory scaffold: never helps at long horizons, hurts 6/10 models

BENCHMARK DOMAINS:
- SE (Software Engineering): Python code editing, bug fixes, refactoring
- WR (Web Research): multi-step information gathering and synthesis
- DP (Document Processing): structured document transformation

DURATION BUCKETS:
- Short: ⩽ 5 minutes human time
- Medium: 5–30 minutes
- Long: 30–120 minutes
- Very Long: ⩾ 120 minutes

VAF BIFURCATION (Key Finding):
Frontier models (VAF = 2.37–2.60) vs mid/small models (VAF ⩽ 1.26)
High VAF = capability signature, NOT instability signature
Weak models fail uniformly so there's no variance to amplify

MOP PARADOX:
DeepSeek V3 and MiniMax M2.5 = best very-long GDS BUT highest meltdown rates
(19% and 13%). Frontier models attempt aggressive strategies. When they work =
high GDS. When they spiral = entropy spike detected by MOP.
```

```
================================================================================
SECTION 2: PROJECT IDEAS FROM THE PAPER
================================================================================

BEGINNER / PORTFOLIO PROJECTS:

1. Reliability Dashboard for LLM calls
   Wrap any LLM API, run tasks k=3 times, track pass/fail, visualize reliability
   curve. Stack: Python + any LLM API + Streamlit/Gradio

2. GDS Task Scorer
   Define a task with subtasks and weights, have LLM attempt it, score partial
   completions instead of binary pass/fail.

3. Meltdown Detector
   Implement MOP metric — track tool calls, compute sliding-window entropy,
   alert when threshold crossed. Plug into any ReAct loop.

INTERMEDIATE PROJECTS:

4. Task Decomposition Reliability Booster
   Auto-decomposer that splits long tasks into short segments, runs each
   independently, chains results. Up to 41.5 pp reliability gain for some
   models per the paper.

5. Scaffold Comparison Tool
   Implement ReAct vs memory-augmented ReAct, measure whether memory helps or
   hurts on your specific use case.

6. Mini Reliability Benchmark
   Create 10-20 tasks across short/medium/long durations, run 2-3 models with
   k=3 repeats, generate your own RDC plots.

ADVANCED / RESEARCH PROJECTS:

7. Reliability-Aware Model Selector
   Given task duration estimate, auto-select best model from pool based on RDC.
   Short task → cheaper model. Very long task → DeepSeek V3.

8. MOP-Triggered Context Reset Agent ← CHOSEN FOR DEVELOPMENT
   Production agent wrapper that detects meltdown, saves progress, resets
   context, continues transparently.

9. Reliability-Aware Fine-Tuning
   Use GDS and RDC as training signals to fine-tune smaller models for flatter
   reliability curves. Uses LoRA/QLoRA fine-tuning with custom reward functions.
```

10. Domain-Adaptive Reliability Predictor
    Classifier that predicts task domain and expected reliability, routes tasks
    to models with matching strengths.

==
SECTION 3: MOP-TRIGGERED CONTEXT RESET AGENT — FULL BUILD PLAN
==

WHAT YOU'RE BUILDING:
A wrapper layer that sits between the user and any LLM agent. It watches the agent's behavior in real time, detects when it's starting to spiral, saves its progress, resets context, and continues — all transparently. Task decomposition feeds into this by breaking large tasks into checkpointable segments from the start.

CORE CONCEPTS TO INTERNALIZE:

Meltdown is not sudden failure. It's a gradual behavioral transition — the agent starts repeating tool calls, contradicts earlier observations, or loops with minor variations. Detected by watching entropy of tool calls over a sliding window of 5 steps. High entropy = disorganized, non-systematic behavior.

Checkpointing means saving the verified state of the task at the last known good point — what subtasks are done, what files written, what information gathered — so a fresh context can pick up without starting over.

Task Decomposition is the upstream defense. Breaking a long task into short segments before execution means each segment runs within a short-horizon window where models are far more reliable. Meltdown detection then acts as the safety net.

SYSTEM COMPONENTS

COMPONENT 1: Task Intake & Decomposer
- Entry point where user submits a task
- Analyzes task and decides: run directly or break into subtasks?
- Produces a task graph: ordered list of subtasks with description, estimated complexity, criticality weight (GDS weights), and dependencies
- Should be domain-aware (SE / WR / DP) since different domains have very different reliability profiles

COMPONENT 2: Execution Engine
- Runs agent on one subtask at a time using standard ReAct loop
- Emits events at every step: tool called, arguments, result, steps elapsed

- Every event goes to the Monitor in real time
- Maintains step budget per subtask (configurable, paper used 50-70 steps max)

COMPONENT 3: Meltdown Monitor (Heart of the Tool)
Receives step-by-step event stream and runs two checks continuously:

  Entropy Check (MOP from paper):
  - Keeps sliding window of last N tool calls
  - Computes Shannon entropy of that distribution
  - If entropy crosses threshold AND entropy has spiked sharply vs prior window
    → meltdown flagged

  Loop Detection (simpler, more immediate):
  - Same (tool, arguments) pair appearing 3+ times in last 6 steps
  - Flag immediately regardless of entropy

  Early Warning Signal:
  - Track whether H(t) has been increasing for last 3 steps even if threshold
    not crossed yet → "Warning" state
  - Useful for UI and giving Checkpoint Manager a heads-up

  Monitor outputs three states: Healthy / Warning / Meltdown

COMPONENT 4: Checkpoint Manager
Activated on Warning or Meltdown signal.

  On Warning:
  - Silently saves snapshot of current state
  - Agent continues
  - Snapshot is ready if meltdown happens

  On Meltdown:
  - Immediately freezes execution
  - Saves: fully complete subtasks with verified outputs, in-progress subtask
    and partial work, key observations from trajectory, current workspace state

  Checkpoint format must be rich enough for fresh context to understand where
  things stand WITHOUT needing full conversation history that caused meltdown.

COMPONENT 5: Context Reset & Resumption
- Constructs fresh system prompt when checkpoint is loaded
- Includes: concise summary of overall task, clear statement of what's complete,
  specific subtask to resume from, critical observations (NOT full history)
- Key insight from paper: memory scaffolds hurt because they dump too much
  context. You want distilled handoff, not a dumped scratchpad.
- Hardest part to get right: too little = agent repeats work, too much =
  recreates the bloated context that caused meltdown

```
COMPONENT 6: GDS Scorer
- Runs after each subtask completes (successfully or after max retries)
- Scores partial completion using criticality weights from task graph
- Gives running GDS throughout execution
- Answers: "even if this task didn't fully complete, how much value did the
  agent deliver?"

COMPONENT 7: Observability Layer
Everything feeds into unified log and optionally a live dashboard.

  Track per session:
  - Full trajectory with tool calls and results
  - Entropy curve over time (see meltdown approaching)
  - How many resets occurred and at what steps
  - Final GDS per subtask and overall
  - Whether decomposition was used and how many segments ran

  This turns the tool from a black box into something practitioners trust.

------------------------------------------------------------
DATA FLOW (END TO END)
------------------------------------------------------------

User submits task
      ↓
Task Decomposer → Task Graph (ordered subtasks with weights)
      ↓
For each subtask:
    Execution Engine runs agent → emits step events
          ↓
    Meltdown Monitor watches events → Healthy / Warning / Meltdown
          ↓
    If Warning:   Checkpoint Manager saves snapshot silently
    If Meltdown:  Checkpoint Manager freezes + saves full checkpoint
                        ↓
                  Context Reset constructs fresh prompt
                        ↓
                  Execution Engine resumes on same subtask
          ↓
    GDS Scorer evaluates subtask completion
          ↓
Move to next subtask
      ↓
Observability Layer logs everything throughout

------------------------------------------------------------
KEY DESIGN DECISIONS
------------------------------------------------------------
```

Entropy window size:
Paper used 5 steps. Smaller = more reactive but noisy. Larger = catches slower spirals but reacts late. Make this configurable.

Reset retry limit:
How many resets on the same subtask before giving up? Paper doesn't address this. 2-3 resets is a reasonable starting point.

Decomposition granularity:
How small should subtasks be? Paper shows short-horizon tasks (<5 min) have ~76% pass@1. That's your target segment size. Finer decomposition = more checkpoint overhead but higher per-segment reliability.

Checkpoint verbosity:
How much of the agent's observations carry into fresh context? Less is more (paper's memory scaffold finding) but you need enough for continuity.

Domain awareness:
Build in SE / WR / DP detection from day one. Different domains need different entropy thresholds and decomposition strategies.

BUILD ORDER (SUGGESTED PHASES)

Phase 1 — Core Loop:
Execution engine + meltdown monitor only. Get entropy calculation and loop detection working correctly on simple tasks. Validate that meltdown signals correspond to real behavioral failures.

Phase 2 — Checkpoint & Reset:
Add checkpoint manager and context reset. Test on tasks where you deliberately induce meltdown (long SE tasks work well per the paper's data).

Phase 3 — Task Decomposer:
Add decomposition as the upstream layer. Connect task graph to execution engine so it sequences subtasks and feeds GDS scoring.

Phase 4 — Observability:
Add dashboard and logging. Makes the tool usable and trustworthy for others.

==========

SECTION 4: DISTRIBUTION & INTEGRATION STRATEGY

==========

THE CORE PROBLEM:

The tool needs to observe an agent's tool calls in real time. It needs to sit somewhere in the execution path. Three realistic positions:

OPTION 1: SDK WRAPPER / LIBRARY (Most Practical First Step)

People install your package (e.g. pip install mop-agent) and wrap their existing agent code with it. Two or three lines and their agent has meltdown detection.

How it works:
Your library intercepts every LLM API call and every tool call result before it goes back to the agent. It watches the stream, runs entropy calculation, and if meltdown is detected pauses execution, saves checkpoint, triggers reset — all within their existing code.

Who this works for:
Developers building agents with Anthropic SDK, OpenAI SDK, LangChain, smolagents, or any Python-based agent framework. They write the agent, they just wrap it with your layer.

The appeal:
Zero infrastructure. No server to run, no account to create. Just a library. How tools like Weights & Biases and LangSmith got initial traction.

The limitation:
Only works if the developer controls the agent code. Can't wrap Claude Code this way because you don't control its internals.

OPTION 2: MCP SERVER (How You Connect to Claude Code / Codex)

MCP (Model Context Protocol) is how Claude Code and other modern agents expose their tool calls to external systems. Build your tool as an MCP server and Claude Code can connect to it directly.

How it works:
Your MCP server registers as a tool provider. Claude Code sees your tools alongside its normal tools. Every time Claude Code makes a tool call, your server observes it, runs the entropy check, and can inject a "stop and reset" instruction back into the conversation if meltdown is detected.

Who this works for:
Anyone using Claude Code, and increasingly any agent that supports MCP — which is becoming the standard.

The appeal:
Native integration path for the modern agentic ecosystem. You're not fighting the architecture, you're working with it.

The limitation:
MCP integration is newer and slightly more complex to build. Users need to configure Claude Code settings to point at your MCP server. Not zero-friction but becoming the expected way tools integrate.

OPTION 3: PROXY LAYER (Universal but Heavy)

You run a local proxy sitting between the developer's agent and the LLM API. All API calls go through your proxy, which observes everything and can intervene.

How it works:
Developer changes one line — their API base URL points to localhost:8080 instead of api.anthropic.com. Your proxy forwards requests, watches responses, intercepts when meltdown is detected.

Who this works for:
Any agent regardless of framework, including ones the developer doesn't control.

The appeal:
Genuinely universal. Works with anything that makes HTTP calls to an LLM API.

The limitation:
Lots of infrastructure to ask users to run. Trust becomes a concern — people routing API keys through your local server. Latency. Hard sell for open source first release.

★ PREFERRED APPROACH: TWO PHASES (DECIDED)

This is the chosen distribution strategy. The proxy approach is ruled out.

WHY THIS APPROACH WORKS:
It has a natural progression story which matters for open source adoption. People discover the tool through the library, get value immediately, then as they move to more sophisticated setups like Claude Code they find the MCP server already waiting for them. Same tool, same mental model, just a different integration point. Anyone with some knowledge of LLMs will be familiar with both a pip library and MCPs — no learning curve on the distribution side.

Phase 1 — Python Library on GitHub/PyPI:

Target developers building agents with Anthropic or OpenAI SDK directly.
Fastest path to people actually using it and giving feedback. README shows
before/after — here's your agent without MOP, here's it with MOP, three lines
changed. Zero infrastructure for the user. No server to run, no account to
create. How tools like Weights & Biases and LangSmith got initial traction.

Phase 2 — MCP Server:
Once core detection logic is solid and tested, wrap it as an MCP server. This
reaches Claude Code users and the broader agentic tool ecosystem without anyone
needing to touch their agent code. Native integration path for the modern
agentic ecosystem — working with the architecture, not against it.

(Proxy approach ruled out: too much infrastructure, trust concerns around API
key routing, latency issues. Not worth the complexity for an open source tool.)

⚠ CAUTIONS — THINGS TO THINK ABOUT BEFORE WRITING ANY CODE

---

1. NAMING MATTERS MORE THAN PEOPLE ADMIT
   The library name and the concept name should be the same thing or obviously
   related. The paper's term "MOP" is catchy but it's an academic acronym.
   Something like agentwatch, reliwatch, or mop-agent needs to feel like
   something a developer would actually type into a terminal without feeling
   silly. Worth spending time on this early because it's hard to change later.

2. THE THREE-LINE PROMISE IS A COMMITMENT
   If your README says "add meltdown detection in three lines" then it genuinely
   needs to be three lines for the happy path. That means your API design has to
   prioritize simplicity over completeness for the initial release. Advanced
   configuration can be optional kwargs and separate documentation. Do not let
   internal complexity leak into the user-facing interface.

3. PHASE 1 AND PHASE 2 SHARE A CORE
   The entropy calculation, loop detection, checkpoint format, and GDS scoring
   logic should live in a shared core module that both the library and the MCP
   server use. If you build Phase 1 well with that separation in mind, Phase 2
   is mostly wrapping that core in MCP protocol rather than rewriting anything.
   This is the single biggest architectural decision — get the core/interface
   boundary right early.

4. YOUR FIRST REAL USERS WILL TELL YOU WHAT THE TOOL ACTUALLY IS
   You might build this as a reliability tool and discover developers mostly use
   it as a debugging tool — watching the entropy curve to understand why their
   agent failed, not just to prevent it. Stay open to that. The observability
   output might end up being the main product, not the intervention.

5. START WITH THE EVENT SCHEMA
   When ready to move into actual development, the logical starting point is
   designing the event schema — what exactly does one "step event" look like as
   a data structure, because everything else in the system is downstream of that
   decision.

WHAT THE GITHUB REPO LOOKS LIKE

The repo serves three audiences:

1. Developers building custom agents
   Import library, works with existing code, get meltdown detection and
   checkpoint/resume out of the box.

2. Claude Code / MCP users
   Follow setup guide to connect MCP server to Claude Code instance. Monitors
   sessions passively from then on.

3. Researchers
   Run benchmark harness against any model, generate own RDC/VAF/GDS/MOP data,
   extending what the paper started. Valuable contribution to research community
   and gives repo credibility.

THE REAL MOAT: OBSERVABILITY OUTPUT

After every session, users get:
- An entropy curve showing exactly where their agent started spiraling
- Their actual GDS score — how much of the task completed even in failure cases
- How many resets were triggered and whether they recovered value
- A reliability score across repeated runs (the paper's core insight)

That's information nobody currently gets from Claude Code or any other agentic
tool.

THE HEADLINE FRAMING FOR THE TOOL:
"Reliability observability for agents"
— not just "meltdown detection"

The paper's finding: frontier models melt down at 19% on very-long tasks.
Roughly 1 in 5 long runs spirals without this kind of intervention. For anyone
running agents on real work — code refactoring, research synthesis, document
pipelines — that's a significant silent failure rate that this tool directly
addresses.

```
529
530 ================================================================================
531 END OF CONVERSATION EXPORT
532 ================================================================================
533
```